

Unit-II (Two Marks)**1. Define Program.**

Program is a set of instructions to solve the given problem by the computer and writing a program is called coding. Computer programs (also software programs or just programs) are organized list of instructions for a computer, when executed, causes the computer to behave in a predetermined manner.

2. Name the problem solving techniques.

Algorithm, Flowchart and Pseudocode are problem solving techniques.

3. List the steps involved in Program Development Cycle.

Development cycle of a program includes Analyse the Problem, Task Analysis, Designing Phase, Algorithm Testing, Coding, Test & Debug the program, Documentation and Implementation.

4. Define Algorithm.

Algorithm is **a step by step procedure of solving a particular problem/program**. It is like set of English like statements. Algorithm means method or recipe or procedure. In general, Algorithm defines the logic of solving a problem.

5. List the characteristics of a good algorithm.

Each and every algorithm is characterized by five important characteristics namely Input, Output, Definiteness, Finiteness and Effectiveness.

6. What is a Flowchart? Give its classification.

Flow chart is a Pictorial representation of a program. It is also defined **as a diagrammatic representation of an algorithm**. The logical sequence of a problem is solved by using standard symbols. Flowcharts are classified into two types.

1. Program flowcharts
2. System flowcharts

7. What is "Pseudocode"?

The Pseudocode is neither an algorithm nor a program. It is an abstract form of a program. It consists of English like statements which perform specific operations. It is defined for an algorithm.

8. What are the types of Programming Languages?

There are many programming languages -- C, C++, Pascal, BASIC, FORTRAN, COBOL, and LISP are just a few. These are all high-level languages. One can also write programs in low-level languages called assembly languages, although this is more difficult. Low-level languages are closer to the language used by a computer, while high-level languages are closer to human languages.

9. What is Debugging?

The process of detecting and correcting errors (mistakes) in the program is known as **debugging**. There is a program called the **debugger** that takes an object program as input and executes it and helps in eliminating the mistakes that occur in the source program.

10. How did the term debug come?

The term debug was coined after detecting a real bug in **MARK-I**, an electro-mechanical computer, in 1945, by Admiral Gracehopper. She removed the bug from the machine and wrote it in a record – “*The MARK-I was debugged today*”.

11. List the type of errors faced by programmers.

- a) Syntax errors
- b) Logical errors
- c) Run-time errors

12. What is testing?

The process of executing the program to test the correctness of the outputs (results) of the problem is called testing. The program is tested by executing with different sets of data. Logical errors are the outcome of this process.

13. What is C language?

The C programming language is a standardized programming language developed in the early 1970s by Dennis Ritchie for use on the UNIX operating system. It has since spread to many other operating systems, and is one of the most widely used programming languages. C is prized for its efficiency, and is the most popular programming language for writing system software, though it is also used for writing applications.

14. How is the name C derived?

C is derived from two early programming languages such as **BCPL** (Basic combined Programming Language) and **B** language. The BCPL was developed by Martin Richards and B was developed by Ken Thompson.

15. What is a byte?

- A byte is a Sequence of 8 bits
- It is the basic unit of computer memory
- 1024 bytes make up one kilobyte (KB), 1024*1024 bytes will make one megabyte (MB) and similarly 1024*1024*1024 bytes will make one Gigabyte (GB)

16. What is a word?

A word is a sequence of 16-bits or a group of two bytes.

17. What is nibble?

- A nibble is a sequence of 4 bits or a half-a-byte
- Memory is not measured in nibbles

18. What is the acronym for ASCII, ANSI?

ASCII - American Standard Code for Information Interchange

ANSI - American National Standard Institute

19. Give the salient feature of C language?

- C language is both high-level and low-level language
- It is used for developing both application and system software
- It has rich set of operators and data types
- Pointers are the powerful concepts that C supports

20. What is a statement?

A statement is an instruction to the computer to perform a specific operation. It may be a declaration, an input-output action, arithmetic and logical operation, assignment or it may be a control statement.

21. What are header files?

Files that are placed at the header portion (before **main ()**) of a C program are called the *header files*. Normally, the standard header files are having **.h** as extension which designates a header file.

22. Define C Tokens and its types.

The basic and the smallest units of a C program are called C *tokens*. There are six types of tokens in C. They are:

Keywords, Identifiers, Constants, Strings, Operators and Special symbols

23. What is an identifier?

An identifier is the name given to the elements of a C program. A variable, an array, a function, a pointer or a structure in C program are identifiers.

24. What is a Keyword?

A keyword is a reserved word in a programming language and whose meaning is predefined. Therefore, a keyword cannot be used as an identifier. The *int, float, void, enum, struct* etc are some of the C keywords.

25. Define Constant and different types of C constants.

The quantity which does not change during the execution of a program is known as *constant*. It remains same throughout the program. There are two types of constants in C. They are:

- Numeric Constants
 - Integer Constants
 - Floating-point Constants
- Non-numeric Constants
 - Character Constant
 - String Constant

26. What is data type?

Data types indicate the type of data that a variable can hold. The data may be numeric or non-numeric in nature.

Eg: long int a; float number; char dept[3];

Where *long int, float, char* are data type specifier.

a ,number,dept are variable name.

27. Mention the different data types of C language?

C supports basically three data types.

- Built-in data types / Primary data types / Fundamental data types
 - Integral data type- *int, char*
 - Floating-point data types- *float, double*
 - Non-specific- *void*
- User-defined data types
 - Type definition- *typedef*
 - Enumerated data types- *enum*
- Derived data types
 - Arrays
 - Pointers
 - Structures and Unions
 - Functions etc

28. What is a variable and how you declare a variable in C?

A variable is an identifier in C. It designates some memory location at which the data is stored. A variable can be declared in C as follows

datatype variableName;

Example: `int a;`
 `float c, f1;`

29. What is a Volatile Variable?

A variable is volatile if the value gets changed at any time by some of the external sources.

Eg: `volatile int num;`

When we declare a variable as volatile the compiler will examine the value of the variable each time it is encountered to see if an external factor has changed the value.

30. What is the need for Type conversion?

In some applications, we may often want to change the data type of the variables. When we declare some variable as **int**, the desired output may be **float** or vice-versa. In such situations, we may change the nature of data stored in a variable. This process is known as *data type conversion*. It is also called the *type casting*. The general form of *type casting* is as follows,

(data type) variable;

31. List the types of Type conversion.

Type Conversion is mainly of two types. They are

- (i) Implicit type conversion (ii) Explicit type conversion

32. What is an Operator and Operand?

Operators are used to carryout the arithmetic and logical operations. Operators act as connectors. They specify the type of operation to be carried out. The values that can be operated by these operators are called *operands*.

Eg: `c=a+b;` `c=a>>4;` `c=5&6;`

Where a, b, c, 4, 5, 6 are *operands*.

`+`, `>>`, `&` are *operators*.

33. How do you classify C operators?

C operators are basically classified into three types.

- Unary operators: -, !, ~, ++, --
- Binary operators
 - Arithmetic operators: +, -, *, / and %
 - Logical operators: &&, ||
 - Relational Operators: <, <=, >, >=, !=, ==
 - Bitwise operators: &, |, ^, <<, >>
- Ternary operator: *Conditional operator* - ?:
- There are some special operators such as *comma operator*, *sizeof() operator*, *pointer operators* (. AND ->)

34. List the precedence of operators.

1. Outermost Parenthesis is evaluated first.
2. Then innermost parenthesis.
3. If there is two or more parenthesis, then the order of execution is from left to right.
4. Next Multiplication and Division are performed.
5. Finally Addition and Subtraction.

35. Explain conditional operator with example.

Conditional Operator Ternary operator is also known as "Ternary operator". The general form of Conditional Operator is

$(exp1)? (exp2) : (exp3);$

where exp1 is the condition which is to be checked and exp2 is the true value and exp3 is the false value. If the condition in exp1 is false then statement in exp3 will be automatically executed.

Eg:

```
#include<stdio.h>
void main()
{
    int x,y,z;
    clrscr();
    printf("Enter the value of a and b :");
    scanf("%d %d",&x,&y);
    z=((x>y)?x:y);
    printf("The biggest value is %d",z);
    getch();
}
```

Output:

Enter the value of a and b: 125 100

The biggest value is 125

36. What is the use of size-of operator?

This operator is used to return the size of string or character. It cannot be used in together with Integers. The syntax is

$sizeof(variable-name);$

37. What is an lvalue?

An lvalue or *location value* is an expression to which a value can be assigned. The lvalue expression is located on the left side of an assignment statement, whereas an rvalue is located on the right side of an assignment statement. Each assignment statement must have an lvalue and an rvalue. The lvalue expression must reference a storable variable in memory. It cannot be a constant.

38. Explain the meaning of Lvalue?

Lvalue or **location value** is an address that is stored into.

In the assignment statement, **a=10;**
we have the **lvalue** of **a** used on the left hand side and the integer constant value 10 assigned to this location.

In the assignment statement, **b=a;**
we have the **rvalue** of **a** (i.e. 10) used to assign to the **lvalue** of **b**. So depending on which side of an assignment, we use the variable **a**, we get two different behaviors.

39. What is the need for input-output functions and list its types?

C is a functional programming language. Therefore, it has no exclusive built-in functions to perform the basic input-output operations. To perform these input-output operations C provides a library of functions. This library is called a *standard input-output library*. It is denoted by **stdio**. The header file containing such library functions is called **stdio.h**

There are two types of input-output (I/O) function. They are:

- Formatted I/O functions
 - printf ()
 - scanf ()
- Unformatted I/O functions
 - getchar () putchar ()
 - gets () puts ()

40. What value is returned when the scanf () function is executed?

The **scanf ()** function returns an **int** value which specifies the number of input fields successfully scanned, converted and stored.

41. What value is returned when the printf() function is executed?

The **printf ()** returns the number of bytes output to stdout.

42. What is the difference between getche () and getch ()?

The **getche ()** function gets a character from the keyboard and echoes to the screen. Whereas, the **getch ()** function gets a character from the keyboard and does not echo to the screen.

43. Difference between scanf and printf function.

Scanf	Printf
Used to accept data	Used to display data
Control string and & operator is used	Control string only used.
It end with semicolon	It end with semicolon
Number of input specified with format String and enclosed within double quotes.	Number of input specified with format string and separated by commas.
The Input variables are specified using address operator (&) is separated by commas.	The output variables are specified by their name and separated by commas.

UNIT 2

2.1. PROGRAM: A program consists of a series of instructions that a computer processes to perform the required operation.

It also contains some fixed data, required to perform the instructions, and the process of defining those instructions and data.

A programmer must determine three basic requirements:

1. The instructions to be performed
2. The order in which those instructions are to be performed.
3. The data required to perform those instructions.

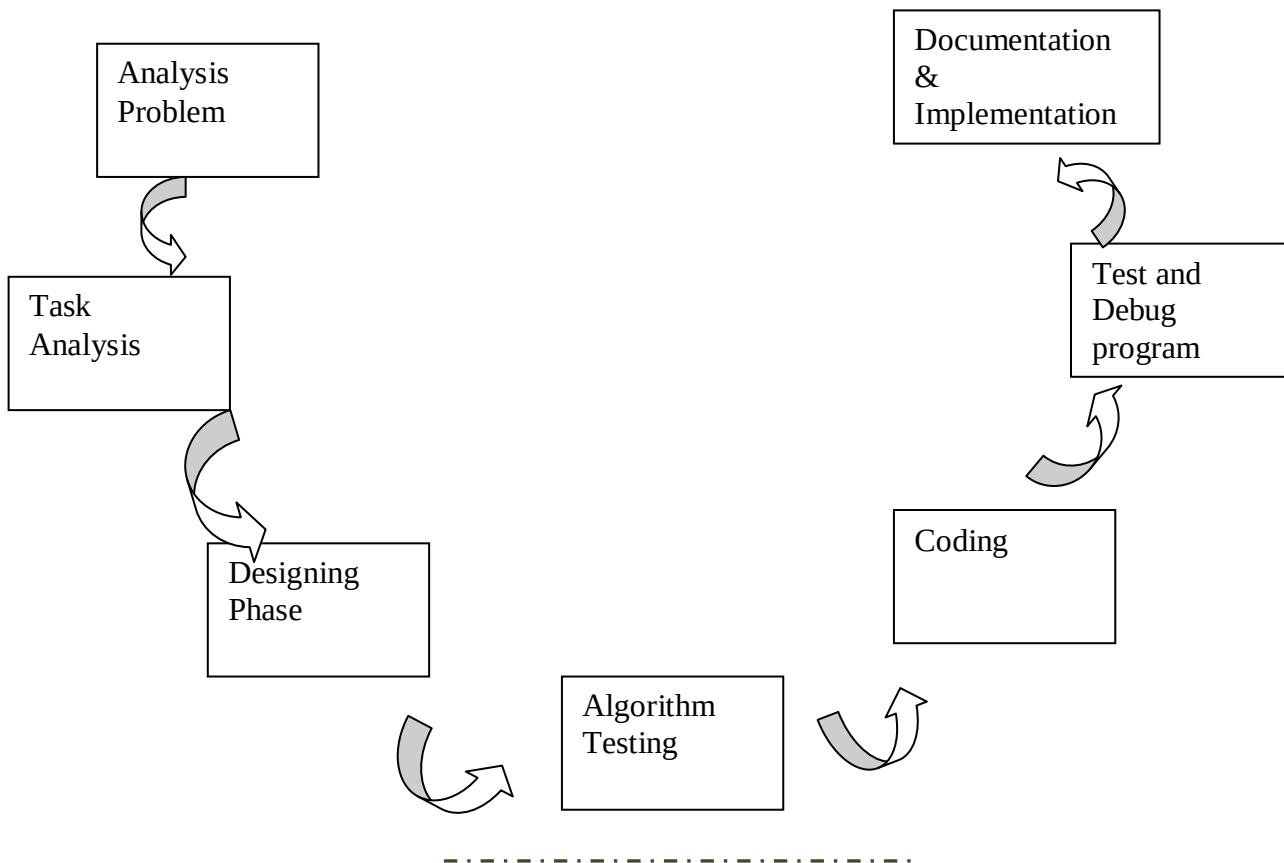
A, B	C=A+B	C
Input	processing	Output

2.2. PROGRAM DEVELOPMENT CYCLE: One common technique used before writing any program development cycle, with the number of steps that may vary according to the person who has formalized the development.

Development cycle of a program includes the following phases:

1. **Analyse / Define the problem:** The problem is analyzed precisely and completely. Based on understanding, the developer knows about the analysis of the problem.
2. **Task Analysis:** The developer needs to develop various solutions to solve the given problem. From these optimum solutions is chosen, which can solve the problem comfortably and economically.
3. **Designing Phase:** After selecting the appropriate solution, algorithm is developed to depict the basic logic of the selected solution. An algorithm represent the problem in logical steps. Further flowcharts and pseudo codes are also developed
4. **Testing the Algorithm for Accuracy:** Before converting the algorithms into actual code, the testing is needed to rectify logical errors. The testing ensures that the algorithm is a true one and it should work for both normal as well as unusual data.
5. **Coding:**
After crossing all the design considerations, the actual coding of the program takes place in the chosen programming language. It may be machine, assembly or high-level languages.
6. **Test and Debus the program:** It is common for the initial program code to contain errors. A program compiler and programmer-designed tests the code for syntax errors. The final result is cross-checked manually calculated output.

7. **Documentation and Implementation:** Once the program is free from all the errors, it is the duty of the program developers to ensure that the program is supported by suitable documentation. The Documentation contains version of the software , help, instructions (user manual), license details, etc.....



2.3. PROBLEM SOLVING TECHNIQUES: Problem solving technique is a set of techniques and graphical tools that help in providing logic for solving a problem

- Some of the **problem solving tools** are
 1. Algorithm
 2. Flowchart
 3. Pseudo code

2.4. ALGORITHM: Algorithm is defined as a finite sequence of explicit instructions that, when provided with a set of input values produces an output and then terminates.

1. In algorithm, Solution for the problem is identified with a finite number of steps.
2. It have **repeated steps**(iterate) until the task is completed
3. For complex problem, more **than one algorithm** are required.
4. Some of the terms associated with algorithm are
 - i. Time complexity
 - ii. Space
5. **Time complexity:** it specifies the amount of time required by an algorithm for performing the desired task.

6. **Space complexity:** it specifies the amount of memory space required by an algorithm for performing the desired task.
7. The algorithm that takes **less time and requires less memory space** is the best one

Characteristics or properties of an algorithm:

1. The instructions must be in an **ordered form**.
2. The instruction must be **simple and concise (crisp)**.
3. There must be **no ambiguity**(doubt) in any instruction
4. It should conclude after a **finite number of steps**.
5. Repetitive programming constructs must possess an **exit condition**, otherwise the program might run infinitely
6. The algorithm must be complete and solve the given problem.

Qualities of a good algorithm:

1. It should use the most **efficient logic** to solve the given problem. (time complexity)
2. It should use **minimal system memory** for its execution. (space complexity)
3. It should generate the **most accurate results**
4. It should be **easy to implement** in the form of a program
5. It should be designed with **standard conventions** so that others are able to easily modify it while adding additional functionality

Sample Algorithm: (to add two integer)

Step 1: Start

Step 2: Accept the first integer as input from the user. (integer 1)

Step 3: Accept the second integer as input from the user . (integer 2)

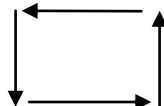
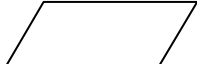
Step 4: Calculate the sum of the two integer. (integer 3 = integer 1 + integer 2)

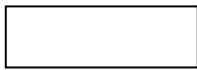
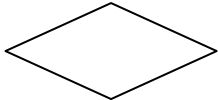
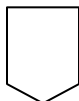
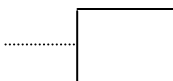
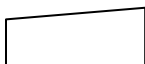
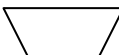
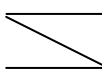
Step 5: Display the result. (integer 3)

Step 6: Stop

2.5. FLOW CHARTS: Flowchart is a pictorial representation of an algorithm in which the steps are drawn in the form of different shapes of boxes and the logical flow is indicated by interconnecting arrows.

1. The **boxes** represent **operations** and the **arrows** represent the **sequence** of operations.
2. Flowchart is drawn according to **defined rules and using standard flowchart symbols** prescribed by ANSI (American National Standard Institute)
 - some standard symbols are...

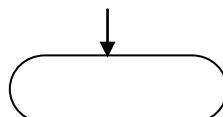
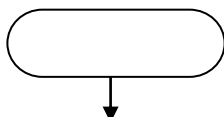
Symbol	Symbol Name	Description
	Flow Lines	• Flow lines are used to connect symbols. • These lines indicate the sequence of steps and the direction of flow of control.
	Terminals	• This symbol (rounded rectangle) is used to represent the Start and Stop operation in the program logic.
	Input / Output	• This symbol (Parallelogram) represent getting input (given by the user to the process and output (given by the process to the user)

	Processing	Process symbol is used for representing arithmetic and data operation
	Decision	Decision symbol denotes a decision to be made. This symbol has one entry and two exit paths. The path depends on the decision process
	Connector	Connector symbol is used to join different flow lines
	Off-page connector	This symbol is used to indicate that the flow chart continues on the next page.
	Document	Document is used to represent a pape document produced during the flowchart process
	Annotation	It is used to provide additional information about flowchart symbol. The content may be in the form of descriptive comments
	Manual input	Manual input symbol represents input to be given by a developer/ programmer
	Manual Operation	Manual operation symbol shows that the process has to be done by a developer / programmer
	Online storage	This symbol represents the online data storage such as hard disks, magnetic drums or other storage devices
	Offline storage	This symbol represents the offline data storage such as data on OCR and data on punched cards
	Communication link	This symbol is used to represent data received or to be transmitted from an external system
	Magnetic disk	This symbol is used to represent data input or output from and to a magnetic disk

Guidelines for preparing flow charts:

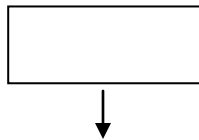
Following guidelines should be used for creating a flowchart:

- The flowchart should be **clear, neat and easy** to follow
- The flowchart must have a **logical start and finish**

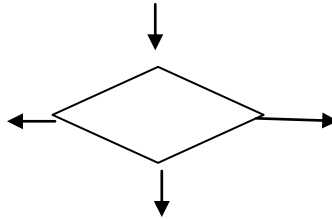


- Only one flow line** should come out from a **process symbol**

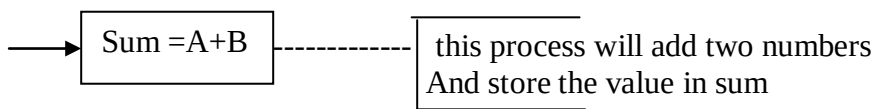




- **Only one flow line should enter a decision symbol.** However two or three flow lines may leave the decision symbol



- if necessary, use annotation symbol to describe data or process



- In case of complex flowcharts, **connector symbols** are used to reduce the number of lines

Benefits or advantage of flowcharts:

The reasons for using flowcharts as problem solving tool are given below

- **Makes logic clear:** The main advantage of using a flow chart to **plan a task**, which makes the logic easier to follow.
- **Communication:** flowcharts are better way of communicating the logic of a system to all concern.
- **Effective analysis:** Each problem can be analyzed in an effective way.
- **Useful in coding:** Flowchart acts as a **guide or blueprint** during the analysis and program development phase. Once the flowchart is ready, the programmers can plan coding process effectively as they know where to begin and where to end, making sure that no steps are omitted
- **Proper testing and debugging:** Flowchart helps in **detecting the errors** in a program, as the developers know exactly what the logic should do.
- **Appropriate documentation:** Flowcharts serve as a good program **documentation tool**.

Limitation or disadvantage of flowchart:

- **Complex:** when a program is very large the flowcharts may continue for many pages, making them hard to follow
- **Costly:** If flowchart is to be drawn for a huge application, the time and cost factor of program development may get out of proportion, making it a costly affair.
- **Difficult to modify:** Due to symbolic nature, any change or modification to a flowchart usually requires **redrawing the entire logic again**, redrawing a complex flowchart is not a simple task.
- **No update:** Usually programs are updated regularly. Updating a flowchart in case of large program is difficult.

2.6. PSEUDOCODE: Pseudocode is made up of two words: pseudo and code. Pseudo means imitation, code refers to instructions, written in a programming language.

- 1.
2. It **is not a real programming code**, but it models and may even look like programming code.
3. Simply pseudocode is the **outline of a program**.
4. Pseudocode uses plain **English statements** rather than symbols, to represent the processes of a computer program. It is known as **PDL (program Design Language)**
5. Pseudocode uses some **keywords** to denote programming processes. Some of them are...
 1. Input: **READ, OBTAIN, GET and PROMPT**
 2. Output: **PRINT, DISPLAY and SHOW**
 3. Compute: **COMPUTE, CALCULATE and DETERMINE**
 4. Initialise: **SET and INITIALISE**
 5. Add one: **INCREMENT**

Guidelines for writing pseudo code:

- Statement should be written in **simple English** and should be programming. language independent.
- It must describe the login plan to develop a program but not programming
- Steps must be **understandable**
- Each instruction should be written in a **separate line** and that must do **one action**.
- **Capitalize keywords** such as READ, GET, etc., should be used.
- Each set of instruction is written from **top to bottom**, with only **one entry and one exit**.
- It should allow for **easy transition** from design to coding

Benefits or advantage of pseudo code:

- Since it is **language independent**, it can be used by most programmers. It allows the developer to express the design in plain natural language
- It is **easier to develop a program** from a pseudocode than with a flowchart. Programmers do not have to think about syntaxes, they can concentrate on logic
- Pseudocode is compact and does not tent to run over many pages. Its **simple structure and readability makes it easier to modify**.

Limitation or disadvantage of pseudo code:

- It does not provide **visual representation** of the program logic
- There is **no accepted standards** for writing pseudocodes. Programmers use their **own style** of writing pseudocode
- Pseudocode cannot be **compiled nor executed**, and there is no **real formatting or syntax rules**.

Sample pseudocode (to add two integer)

Define: Integer num1, num2, result.

Input: **GET** num1 from user

Input: **GET** num2 from user

COMPUTE the sum with num1 and num2

Output: **DISPLAY** sum

.....

2.7. INTRODUCTION TO C

1. C is a popular **general purpose programming language**.
2. C language has been designed and developed by **Dennis Ritchie at Bell Laboratories in 1972**.
3. It is an offspring of the “**basic combined programming language**” called B developed in the year **1960** at Cambridge university.
4. This language is associated closely with **unix operating system**. The source code for the unix operating system is coded in C.
5. C runs under a number of operating systems including MS-DOS.
6. C program are **efficient, fast and highly portable**, that is C program written on one computer can be run on another with little or no modification.
7. C function are the **building block** of a program.
8. C combines the features of **high-level language** and **low level language**, that is why it is known as **middle level language**.
9. It is well suited for writing both **application software and system software**.
10. C is a **structured language**.
11. A structured language allows variety of programs in small modules.
12. It is easy for **debugging, testing and maintenance** if a language is a structured one.
13. Some of the *structured language* are **C,Pascal and Fortran**. Some of the *non-structured language* are **Basic and Cobol**.
14. C provides loop constructs like while, do-while and for.

Structure of a C program:

1. C program contains a number of **building blocks** known as functions.
2. Each function defined in C performs **a specific task**
3. A function is **subroutine that may consist of one or more statements**.
4. A C program comprises the following different sections

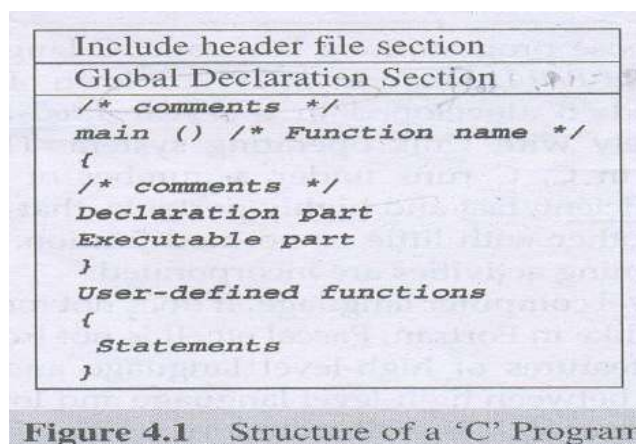


Figure 4.1 Structure of a 'C' Program

Include header file section:

1. C program depends upon some header files for function definition that are used in program.
2. Each header file by default is extended with **.h**.
3. Header file should be included using **#include** directive as given here.
4. **#** - preprocessor

For ex: **#include <stdio.h>** or **#include "stdio.h"**

Global declaration:

5. Declaring some variables that are used in more than one function.
6. These variables are known as **global variables**.
7. This section must be declared outside of all the functions.

Main function:

8. Every C program must contain **main()** function
9. Empty parenthesis after main is necessary.
10. Execution of the program always begins with the function **main()**.
11. The program execution starts from the opening brace (**{**), ends with the closing brace (**}**).
12. Between these two braces the program should declare the **declaration and executable part**.

Declaration part:

1. The declaration part declares the entire variables that are used in executable part.
2. The initialization of variables is also done in this section.
3. Initialization means providing **initial value** to the variables.

Executable part:

It contain set of statements

User-defined function:

The function defined by the user are called user defined functions

Comments

1. Comments are not necessary in the program
2. To understand the flow of the program the programmer can include comments in the program.
3. Comments are useful in documentation. **/*, */** called delimiters.

Eg: **/*** this is single comments ***/**

/*this is an example of **/*** nested comments***/**

```
// single comments
/* this is an example of
Comments in
Multiple lines*/
```

Programming Rules:

- All statements should be written in **lower case letters**. Upper case letters are only used for symbolic constants
- **Blank spaces** may be inserted between the words. This improves the readability of the statements, however, it is not used while declaring a variable, keyword, constant and functions.
- All the statements must **end with semi colon (;)**. Two or more statements can be written in single line separated by semi colon.
- The opening and closing braces should be **balanced**.

Executing the program:

1. Creation of program:

Program should be written in **C editor**. The default extension is **C**.

2. Compilation and linking of a program:

The source program should be translated into object program which is suitable for execution by the computer. Compilation process translate the source program are stored in another file with the same file name with extension **.obj**.

Linking is the process of linking the program files and functions together that is required by the program. For ex: if **pow()** function is used the object code of this function should be brought from **math.h**.

3. Executing the program:

After the compilation the executable object code will be loaded in the computers main memory and the program is executed.

The C character set

All programs must confirm to rules pre-defined in C Language.

The character set in C Language can be grouped into the following categories.

- 1.Letters
- 2.Digits

3.SpecialCharacters

4. White Spaces

White Spaces are ignored by the compiler until they are a part of string constant. White Space may be used to separate words, but are strictly prohibited while using between characters of keywords or identifiers.

Table 5.1 Character Set

Letters	Digits	White Spaces
Capital A to Z	All decimal digits 0 to 9	Blank space
Small a to z		Horizontal tab
		Vertical tab
		New line
		Form feed

Table 5.2 List of Special Characters

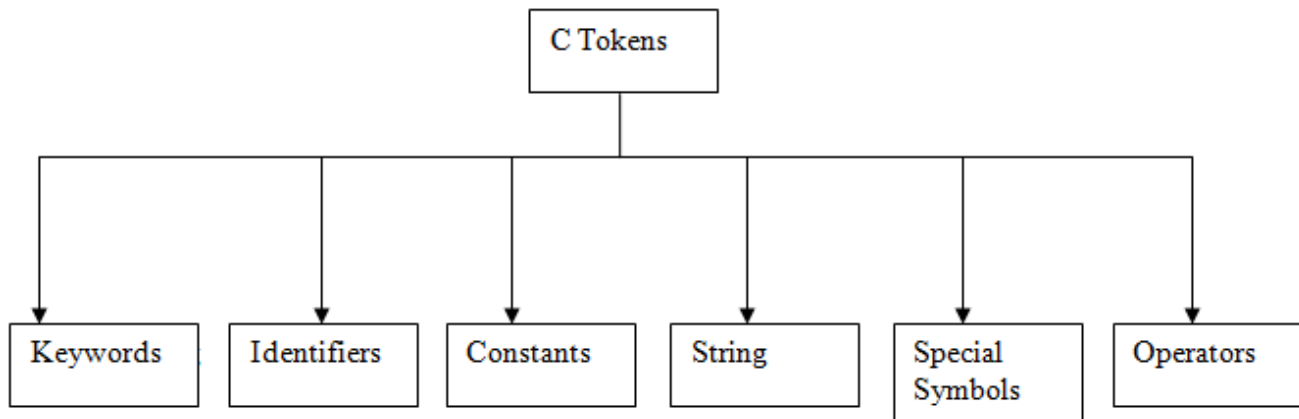
,	Comma	&	Ampersand
.	Period or dot	^	Caret
;	Semicolon	*	Asterisk
:	Colon	-	Minus
'	Apostrophe	+	Plus
"	Quotation mark	<	Less than
!	Exclamation mark	>	Greater than
	Vertical bar	()	Parenthesis left /right
/	Slash	[]	Bracket left/right
\	Back slash	{ }	Braces left/right
~	Tilde	%	Percent
_	Underscore	#	Number sign or Hash
\$	Dollar	=	Equal to
?	Question Mark	@	At the rate

Delimiters:

Table 5.3 Delimiters

Delimiters	Use
: Colon	Useful for label.
; Semicolon	Terminates statement.
() Parenthesis	Used in expression and function.
[] Square Bracket	Used for array declaration.
{ } Curly Brace	Scope of statement.
# Hash	Preprocessor directive.
, Comma	Variable separator.

2.8. C TOKENS:



keywords

1. C keywords are reserved words by the compiler.
2. All the c keywords have been assigned a fixed meaning.
3. Keywords cannot be used as variable names because they have been assigned fixed jobs.

Table 5.4 C Keywords

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

Additional keywords for Borland C are as follows.

asm	cdecl	far	huge	interrupt	near	pascal
-----	-------	-----	------	-----------	------	--------

IDENTIFIERS

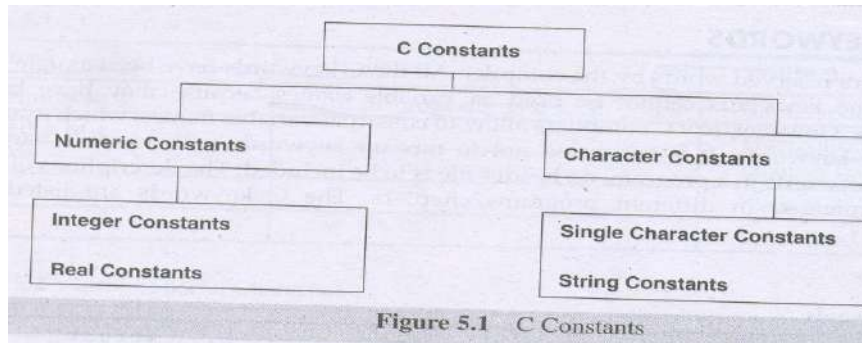
1. Identifiers are names of variables, functions and array.
2. The (_) underscore symbol can be used as an identifiers

Eg: # define N 10, int A

N and A are user defined identifiers

CONSTANTS

1. Constants which do not change during the execution of a program.
2. There are several types o constants in C.
3. C supports several types of constants.



Numeric constants:

1.Integer constants: It consists of a set of digits 0 to 9 preceded by an optional + or - sign. Spaces, commas and non digit characters are not permitted between digits.

Example for valid decimal integer constants is

123

-31

0

562321

+ 78

Some examples for **invalid integer constants** are

15 750

20,000

Rs. 1000

2.Real constants: Real Constants consists of a fractional part in their representation. Integer constants are inadequate to represent quantities that vary continuously. These quantities are represented by numbers containing fractional parts like 26.082. Example of real constants is

0.0026

-0.97

435.29

+487.0

Real Numbers can also be represented by exponential notation. The general form for exponential notation is mantissa exponent. The mantissa is either a real number expressed in decimal notation or an integer. The exponent is an integer number with an optional plus or minus sign.

Character constants:

1.Single Character Constants: A Single Character constant represent a single character which is enclosed in a pair of quotation symbols. Example for character constants are

'5'

'x'

','

''

All character constants have an equivalent integer value which is called ASCII Values.

2.String Constants : A string constant is a set of characters enclosed in double quotation marks. The characters in a string constant sequence may be a alphabet, number, special character and blank space. Example of string constants are

"VISHAL"

"1234"

"God Bless"

"!.....?"

VARIABLES:

1. A variable is a value that can change any time. It is a memory location used to store a data value.
2. Variables can be of different data types. The data types are integers, real or character constants
3. A variable is a data name used for storing a data values.

Rules for defining variables:

1. White space is not allowed
2. The length of a variable must not be more than 8 characters
3. A variable should not be a Keyword
4. The variable names may be a combination of upper and lower characters.
5. The variable name should not start with a digit

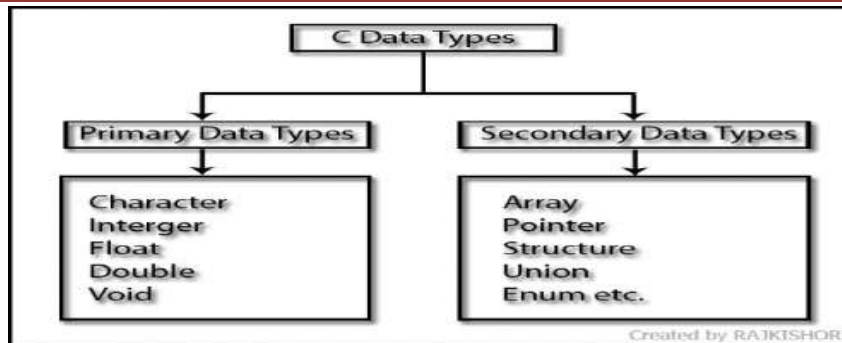
Example of variable names are

- 1.Sun
- 2.number
- 3.Salary
- 4.Emp_name
- 5.average1

Examples of Invalid Variable names are

- 1.123
- 2.(area)
- 3.6th
- 4.%abc

2.9. DATA TYPES: A program usually contains different types of data types (integer, float, character etc.) to store the values being used in the program along with some library function and user defined function (UDF) to process that stored



Different datatypes in C are :-

S.No	Data Type	Size in Bytes	Range	Format Specifier	Description	Syntax	Example
1	char	1	-128 to 127	%c	Character type variable can hold a single character	char <variable name>;	char ch = 'a'; char c = 120;
2	Unsigned char	1	0 to 255	%c	Unsigned characters have values between 0 and 255	Unsigned char <variable name>;	Unsigned char ch = 121
3	Short (or) int	2	-32,768 to 32,767	%i or %d	Integers are whole numbers with a range of values, range of values are machine dependent	int<variable name>;	int num1 = 45; short int num2=-345
4	unsigned int	2	0 to 65535	%u	unsigned integers are always positive and use all the bits for the magnitude of the number	unsigned int<variable name>;	Unsigned int a=65535
5	long or long int	4	-2147483648 to 2147483647	%ld	are used to declare a longer range of values and it occupies 4 bytes of storage space	long <variable name>;	long n=2147483645
6	unsigned long	4	0 to 4294967295	%lu	Unsigned long take only positive numbers and use all the bits for the magnitude of the number	Unsigned long <variable name>;	Unsigned long nn = 4294967295
7	Float	4	3.4 e - 38 to 3.4 e +38	%f or %g	The float data type is used to store fractional numbers (real numbers) with 6 digits of precision.	float <variable name>;	float num1=9.121111
8	Double	8	1.7 e - 308 to 1.7 e +308	%lf	The double is same as float but with longer precision and takes double space (8 bytes) than float	Double <variable name>;	double num2=89.443332221
9	Long double	10	3.4 e- 4932 to 3.4 e + 4932	%lf	To extend the precision further we can use long double which occupies 10 bytes of memory space.	Long double <variable name>;	long double num3;=994080.56790
10	Void	The void type has no values therefore we cannot declare it as variable as we did in case of integer and float.					

Variable initialization

Variables declared can be assigned or initialized using an assignment operator '='.

Syntax:

<data type> <variable name> = Constant
--

Or

<data type> <variable name>

Eg: int y = 2

Int x,y,z;

Type conversion:

Type conversion occurs when the expression has data of mixed data types.

Eg: Void main() { Int a=5, b=2; c=a/b; Printf("the value of C is", c); Printf("(type casting) the value of C is", (float)c); }	Output: The value of C is 2 (type casting) the value of C is 2.5
---	---

.....

2.10. OPERATORS AND EXPRESSIONS: The symbols which are used to perform logical and mathematical operations in a C program are called C operators. Operators, functions, constants and variables are combined together to form expressions.

Types of C operators:

C language offers many types of operators. They are,

1. Arithmetic operators
2. Increment/decrement operators
3. Assignment operators
4. Relational operators
5. Logical operators
6. Conditional operators (ternary operators)
7. Bit wise operators
8. Special operators

1. Arithmetic Operators:

- C Arithmetic operators are used to perform mathematical calculations like addition, subtraction, multiplication, division and modulus in C programs.

S.no	Arithmetic Operators	Operation	Example
1	+	Addition	A + B
2	-	Subtraction	A - B
3	*	multiplication	A * B
4	/	Division	A / B
5	%	Modulus	A % B

Example program for C arithmetic operators:

```
#include <stdio.h>
int main()
{
    int a=40,b=20, add,sub,mul,div,mod;
    add = a+b;
    sub = a-b;
    mul = a*b;
    div = a/b;
    mod = a% b;

    printf("Addition of a, b is : %d\n", add);
    printf("Subtraction of a, b is : %d\n", sub);
    printf("Multiplication of a, b is : %d\n", mul);
    printf("Division of a, b is : %d\n", div);
    printf("Modulus of a, b is : %d\n", mod);
}
```

2. Increment/decrement Operators

- Increment operators are used to increase the value of the variable by one and decrement operators are used to decrease the value of the variable by one in C programs.
- Syntax:

Increment operator : ++var_name; (or) var_name++;

Decrement operator : -- var_name; (or) var_name --;

- Example:

Increment operator : ++ i ; i ++;

Decrement operator: -- i ; i --;

S.No	Operator type	Operator	Description
1	Pre increment	++i	Value of i is incremented first then incremented is assigned to variable i.
2	Post-increment	i++	Value is assigned to the variable first and then assigned value get incremented.
3	Pre decrement	--i	Value of i is decremented first then incremented is assigned to variable i.
4	Post decrement	i--	Value is assigned to the variable first and then assigned value get decremented.

<pre>//example for pre-Increment operator #include <stdio.h> int main() { int i=0; while(++i < 5) { printf("%d ",i); } return 0; }</pre>	<pre>//Example for post decrement operator #include <stdio.h> int main() { int i=10; while(i-- > 5) { printf("%d ",i); } return 0; }</pre>
--	--

3. Assignment Operators

- In C programs, values for the variables are assigned using assignment operators. Other assignment operators in C language are given below.

Operators		Example	Explanation
Simple assignment operator	=	sum=10	10 is assigned to variable sum
Short hand operators	+=	sum+=10	This is same as sum=sum+10
	-=	sum-=10	This is same as sum = sum-10
	=	sum=10	This is same as sum = sum*10
	/=	sum/=10	This is same as sum = sum/10
	%=	sum%=10	This is same as sum = sum%10
	&=	sum&=10	This is same as sum = sum&10
	^=	sum^=10	This is same as sum = sum^10

Example program for C assignment operators:

```
# include <stdio.h>
int main()
{
    int Total=0,i;
    for(i=0;i<10;i++)
    {
        Total+=i; // This is same as Total = Total+i
    }
    printf("Total = %d", Total);
}
```

4. Relational Operators

- Relational operators are used to find the relation between two variables. i.e. to compare the values of two variables in a C program.

S.no	Operators	Description	Example x=5,y=7
1	>	x is greater than y	If(x>y) then return false(0)
2	<	x is less than y	If(x<y) then return true(1)
3	>=	x is greater than or equal to y	If(x>=y) then return false(0)
4	<=	x is less than or equal to y	If(x<=y) then return true(1)
5	==	x is equal to y	If(x==y) then return false(0)
6	!=	x is not equal to y	If(x!=y) then return true(1)

Example:

<pre>#include <stdio.h> int main() { int m=40,n=20; if (m == n) { printf("m and n are equal"); } }</pre>	<pre>else { printf("m and n are not equal"); } }</pre>
--	--

5.Logical Operators

- These operators are used to perform logical operations on the given expressions.
- There are 3 logical operators in C language. They are, logical AND (&&), logical OR (||) and logical NOT (!).

S.no	Operators	Name	Example	Description
1	&&	logical AND	(x>5)&&(y<5)	It returns true when both conditions are true
2		logical OR	(x>=10) (y>=10)	It returns true when at-least one of the condition is true
3	!	logical NOT	!((x>5)&&(y<5))	If “((x>5) && (y<5))” is true, logical NOT operator makes it false

Example:

```
#include <stdio.h>
int main()
{
    int m=40,n=20;
    int o=20,p=30;
    if (m>n && m !=0)
    {
        printf("&& Operator : Both conditions are true\n");
    }
    if (o>p || p!=20)
    {
        printf("|| Operator : Only one condition is true\n");
    }
    if (!(m>n && m !=0))
    {
        printf("! Operator : Both conditions are true\n");
    }
    else
    {
        printf("! Operator : Both conditions are true. " \
            "But, status is inverted as false\n");
    }
}
```


6. Conditional Operators: Conditional operators return one value if condition is true and returns another value if condition is false. This operator is also called as ternary operator.

Syntax : (Condition? true_value: false_value);

Example : (A > 100 ? 0 : 1);

In above example, if A is greater than 100, 0 is returned else 1 is returned. This is equal to if else conditional statements.

Example program for conditional/ternary operators in C:

```
#include <stdio.h>
int main()
{
    int x=1, y ;
    y = ( x ==1 ? 2 : 0 ) ;
    printf("x value is %d\n", x);
    printf("y value is %d", y);
}
```

OUTPUT:

x value is 1
y value is 1

7.Bit wise Operators

- These operators are used to perform bit operations. Decimal values are converted into binary values which are the sequence of bits and bit wise operators work on these bits.
- Bit wise operators in C language are & (bitwise AND), | (bitwise OR), ~ (bitwise NOT), ^ (XOR), << (left shift) and >> (right shift).

Bit wise operators

Operator symbol	Operator name
&	Bitwise AND
	Bitwise OR
~	Bitwise NOT
^	XOR
<<	Left Shift
>>	Right Shift

Truth table

x	y	x y	x & y	x ^ y
0	0	0	0	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	0

Example :

a = 0001000

b= 2

a << b = 0100000

a >> b = 0000010

Example:- #include <stdio.h> int main() { int m=40,n=80,A,O,X,N ; A = (m&n); O = (m n); N = (~m);	X = (m^n); printf("AND_opr value = %d\n",A); printf("OR_opr value = %d\n",O); printf("NOT_opr value = %d\n",N); printf("XOR_opr value = %d\n",X); printf("left_shift value = %d\n", m << 1); printf("right_shift value = %d\n", m >> 1); }
---	---

8.Special Operators

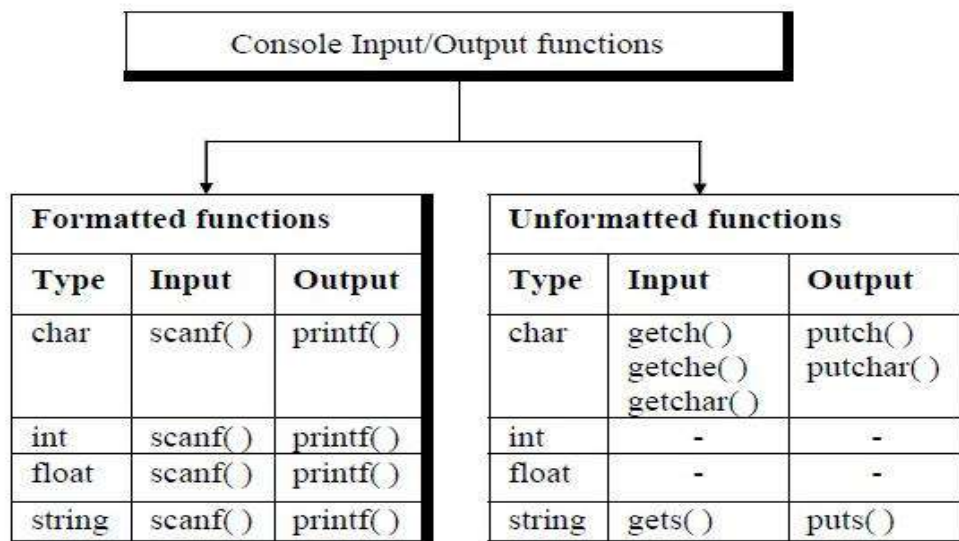
Below are some of special operators that C language offers.

S.no	Operators	Description
1	&	This is used to get the address of the variable. Example : &a will give address of a.
2	*	This is used as pointer to a variable. Example : * a where, * is pointer to the variable a.
3	Sizeof ()	This gives the size of the variable. Example : size of (char) will give us 1.

EXample:

#include <stdio.h> int main() { int *ptr, q; q = 50; /* address of q is assigned to ptr */	ptr = &q; /* display q's value using ptr variable */ printf("%d", *ptr); return 0; }
---	--

2.11. INPUT AND OUTPUT IN C:



There are number of I/O functions in c, based on the data types. The input / Output functions are classified into two types.

1. Formatted functions

2. Unformatted functions

- a) **Formatted functions:** The formatted i/o functions read and write all types of data values. They require conversion symbol to identify the data type.
- b) **Unformatted functions:** The unformatted input/output functions only work with the character data type. They do not require conversion symbol for identification of data types because, they work only with character data type.

The printf() statement: The printf() function prints all types of data values to the console. It requires conversion symbol and variable names to print the data. The conversion symbol and variable names should be same in number.

The scanf() statement: The scanf() statement reads all types of data values. It is used for runtime assignment of variables. The scanf() statement also requires conversion symbol to identify the data to be read during the execution of a program.

Escape sequences with them ASCII Values:

Escape Sequence	use	ASCII Value
\n	New line	010
\b	Backspace	008
\f	Form feed	012
\'	Single quote	039
\\	Backslash	092
\0	Null	000
\t	Horizontal Tab	009
\r	Carriage return	013
\a	Alert(Bell)	007
\"	Double Quote	034
\v	Vertical tab	011
\?	Question mark	063

Three Types of I/O Functions:

Unformatted Functions:

- i) Character I/O
- ii) String I/O
- iii) File I/O

Character I/O:

getchar(): This function reads character type data from the standard input. It reads one character at a time till the user press the enter key.

<pre>#include<stdio.h> Void main() { Char ch[20]; int c=0; while((ch[c]=getchar())!='\n') { c++; } }</pre>	<pre>Ch[c]='\0'; Printf("\n%s",ch); } Output: INDIA INDIA</pre>
--	---

putchar(): This function prints one character on the screen at a time, which is read by the standard input

<pre>#include<stdio.h> main() { char ch[20]; int c=0; printf("Enter text there:"); scanf("%s",ch); printf("\n The Entered text:");</pre>	<pre>while(ch[c]!='\0') { Putchar(ch[c]); C++; } } OUTPUT: Enter text there:PONDY The Entered text:PONDY</pre>
--	--

getch & getche():These functions read any alphanumeric characters from the standard input device. The character entered is not displayed by getch() function.

<pre>#include<stdio.h> Void main() { Printf("Enter any two alphabetic:"); getche(); getch(); }</pre>	<pre>OUTPUT Enter any two alphabetic :m</pre>
--	---

Putch():This function prints any alphanumeric character taken by the standard input device.

<pre>Void main() { Char ch; Printf("Press any key to continue"); ch=getch();</pre>	<pre>printf("\n U pressed:"); putch(ch); } Output: Press any key to continue:Q U pressed:Q</pre>
--	--

String I/O:

gets():This function is used for accepting any string through stdin(keyboard) until enter key is pressed. The header file stdio.h is needed for implementing the above function.

<pre>#include<stdio.h> Void main() { Char ch[30]; Printf("Enter the string:"); gets(ch); printf("\n Entered string:%s",ch); }</pre>	Output: Enter the string:Charles Entered string: Charles
---	---

puts():This function prints the string or character array.

<pre>#include<stdio.h> void main() { char ch[30]; printf("Enter the string:"); gets(ch); puts("Entered String:"); puts(ch); }</pre>	Output: Enter the string:Tommy Dreamer Entered string: Tommy Dreamer
---	---

cgets(): This function reads string from the console. The syntax : cgets(char * st);
It requires character pointer as an argument. The string begins from str[2].
cputs(): This function displays string on the console. The syntax : cputs(char * st);

<pre>#include<stdio.h> void main() { static char *t; printf("\n Enter text here:"); cgets(t); t+=2; printf("\n you entered Text"); cputs(t); }</pre>	OUTPUT: Enter text here: abcd you entered Text : abcd
--	--
